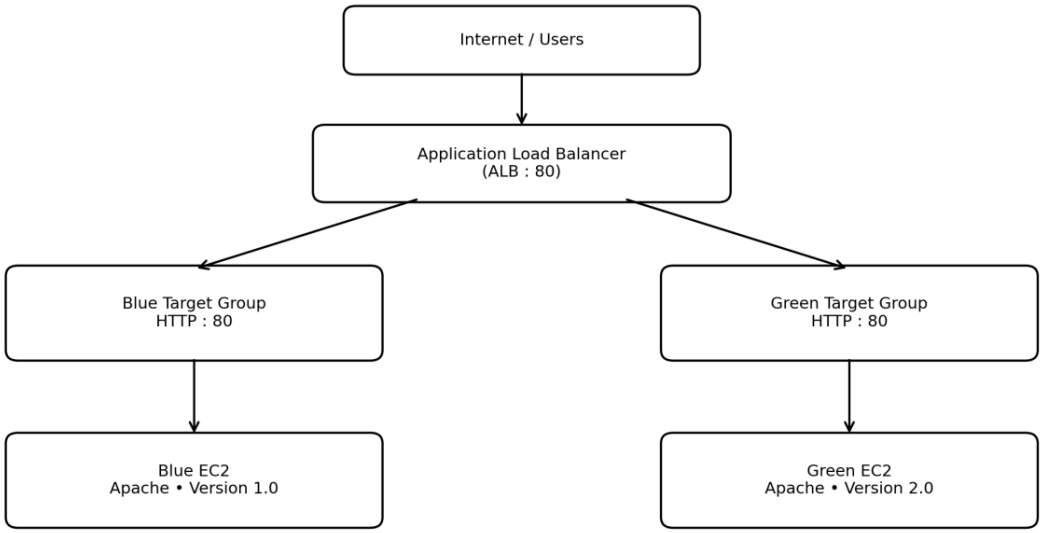
PROJECT REPORT

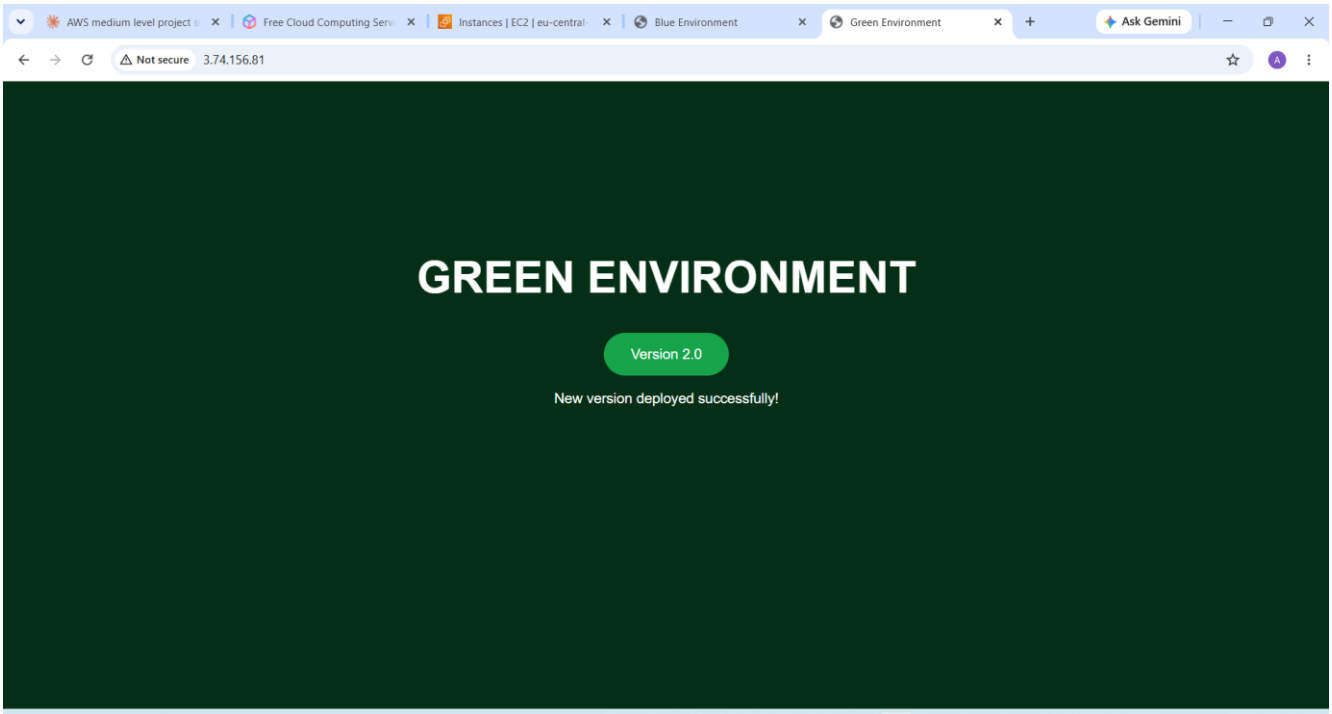
BLUE-GREEN DEPLOYMENT USING AWS EC2 AND APPLICATION LOAD BALANCER

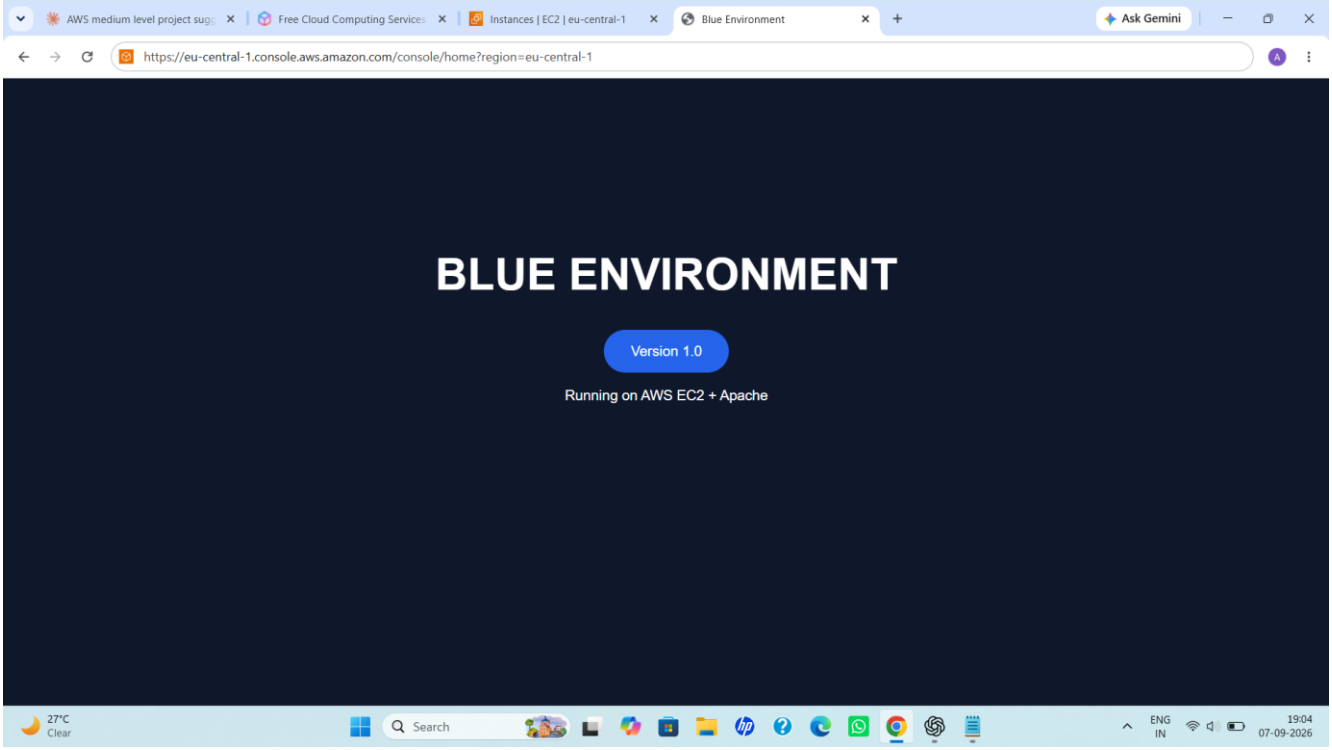
A Cloud / DevOps Deployment Project



Traffic can be switched from Blue → Green and rolled back to Blue.

Prepared by: ATHARVA VADAGE


AWS medium level project
Free Cloud Computing Serv
Instances | EC2 | eu-central-
Blue Environment
Green Environment
Ask Gemini
Not secure
3.74.156.81
GREEN ENVIRONMENT
Version 2.0
New version deployed successfully!
27°C
Clear
Search
ENG
IN
19:17
07-09-2026


AWS medium level project sug
Free Cloud Computing Services
Instances | EC2 | eu-central-1
Blue Environment
Ask Gemini
https://eu-central-1.console.aws.amazon.com/console/home?region=eu-central-1
BLUE ENVIRONMENT
Version 1.0
Running on AWS EC2 + Apache
27°C
Clear
Search
ENG
IN
19:04
07-09-2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my instructor, institution, and everyone who supported me while learning AWS, Linux, networking, and DevOps concepts. This project helped me understand how deployment strategies can improve application availability, release safety, and rollback capability.

ABSTRACT

Blue-Green Deployment is a release strategy in which two separate application environments are maintained: Blue represents the currently stable version, while Green represents the new version. In this project, two Amazon EC2 instances run Apache web servers hosting Version 1.0 and Version 2.0. Separate Application Load Balancer target groups are created for the Blue and Green environments. The Application Load Balancer receives public HTTP traffic and forwards it to the selected target group. After the Green environment is verified as healthy, the listener is switched from Blue to Green. If a problem occurs, traffic can be switched back to Blue, providing a simple and fast rollback mechanism.

The project demonstrates practical AWS services including EC2, Application Load Balancer, Target Groups, VPC networking, Security Groups, and Apache HTTP Server. It also demonstrates important DevOps concepts such as deployment isolation, health checks, controlled release, rollback, and high availability.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Scope of the Project

5. Technologies and AWS Services

6. System Architecture

7. Flowcharts

8. Project Requirements

9. Implementation Methodology

10. Configuration Details

11. Blue-Green Deployment Procedure

12. Testing and Validation

13. Security Considerations

14. Advantages and Limitations

15. Cost and Resource Management

16. Results

17. Future Enhancements

18. Conclusion

19. References

1. INTRODUCTION

In traditional deployments, a new software version may be installed directly over the existing production version. This can cause downtime or make rollback difficult when the new release contains an error. Blue-Green Deployment addresses this problem by keeping two environments available. One environment serves production traffic while the other is prepared and tested for the next release.

This project implements the concept on AWS. The Blue environment hosts a stable web page identified as Version 1.0. The Green environment hosts a new web page identified as Version 2.0. An Application Load Balancer controls which environment receives user requests.

2. PROBLEM STATEMENT

Organizations need a deployment method that reduces application downtime, allows testing of new releases before exposing them to users, and provides a quick rollback path. A direct in-place update can create service interruptions and makes it harder to recover from failed deployments. Therefore, a controlled deployment architecture is required.

3. OBJECTIVES

- Understand the Blue-Green deployment strategy.
- Deploy two application versions on separate EC2 instances.
- Configure Apache HTTP Server on Linux-based EC2 instances.
- Create and configure separate target groups for Blue and Green.
- Use an Application Load Balancer to control application traffic.
- Use target health checks to verify the deployment environment.
- Demonstrate production traffic switching from Blue to Green.
- Demonstrate rollback from Green to Blue.
- Apply security groups using least-privilege network access where practical.

4. SCOPE OF THE PROJECT

The project focuses on a simple static web application deployed on EC2 and exposed through an Application Load Balancer. It is intended as a learning project and can be extended to real application stacks such as Node.js, Python, Java, or containerized services.

5. TECHNOLOGIES AND AWS SERVICES

Component	Purpose
Amazon EC2	Hosts the Blue and Green web servers.
Application Load Balancer	Receives HTTP requests and forwards traffic to the selected target group.
Target Groups	Group and health-check the Blue and Green EC2 targets.
VPC	Provides isolated network infrastructure for the resources.
Security Groups	Control inbound and outbound network traffic.

Amazon Linux 2023	Operating system for EC2 instances.
Apache HTTP Server	Web server serving Version 1.0 and Version 2.0 pages.
HTML/CSS	Used to create the demonstration web pages.
AWS Management Console	Used to create and configure AWS resources.

6. SYSTEM ARCHITECTURE

The architecture contains an internet-facing Application Load Balancer in a VPC. The ALB has an HTTP listener on port 80. The listener forwards requests to either the Blue target group or the Green target group. Each target group contains one EC2 instance running Apache.

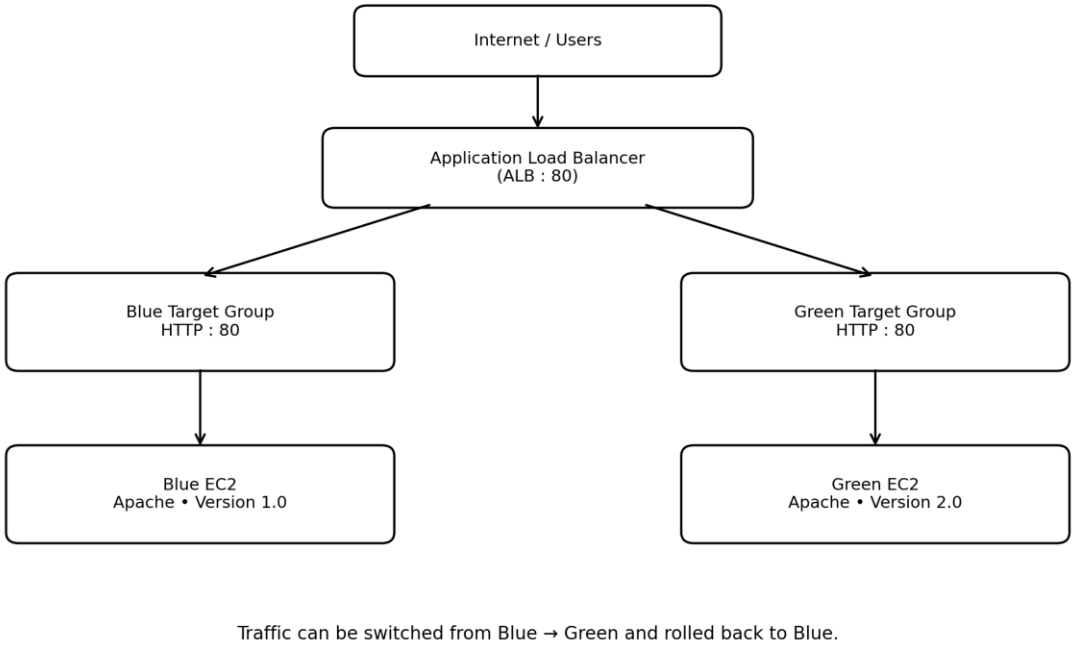


Figure 1: Blue-Green AWS Architecture

7. FLOWCHARTS

7.1 Deployment Flow

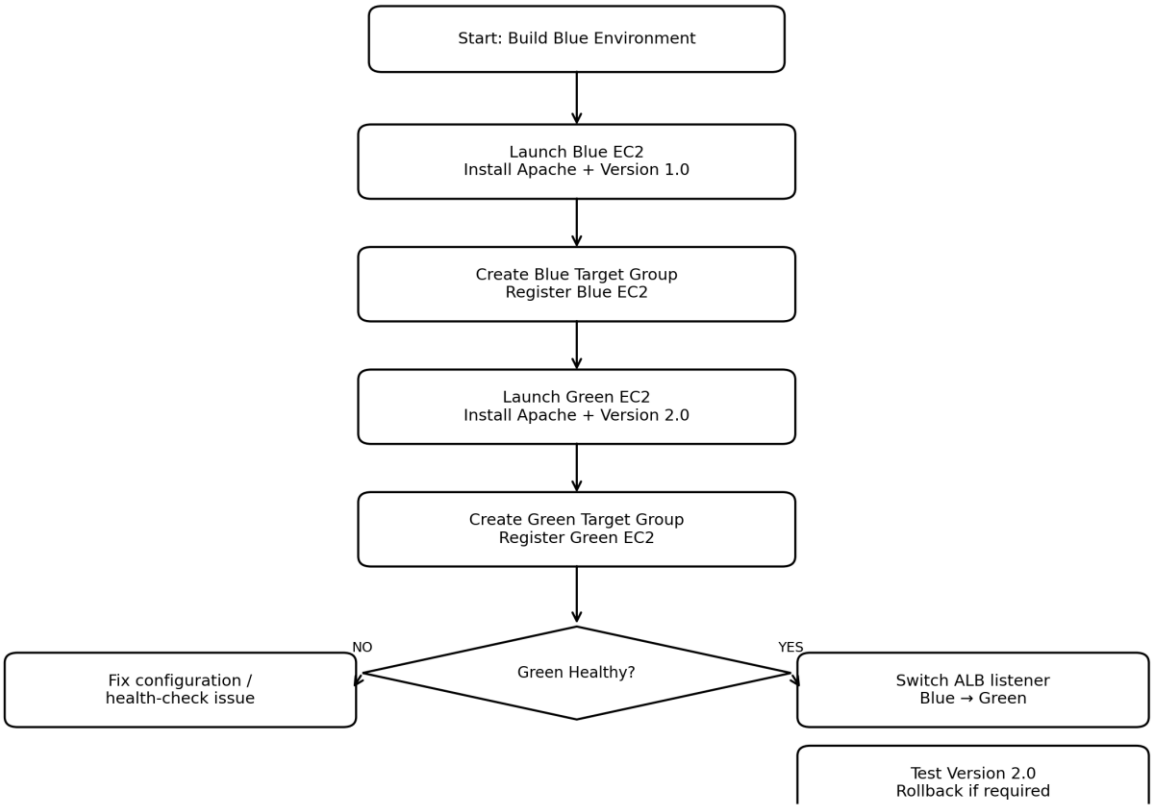


Figure 2: Blue-Green Deployment Workflow

7.2 Traffic Switching and Rollback Flow

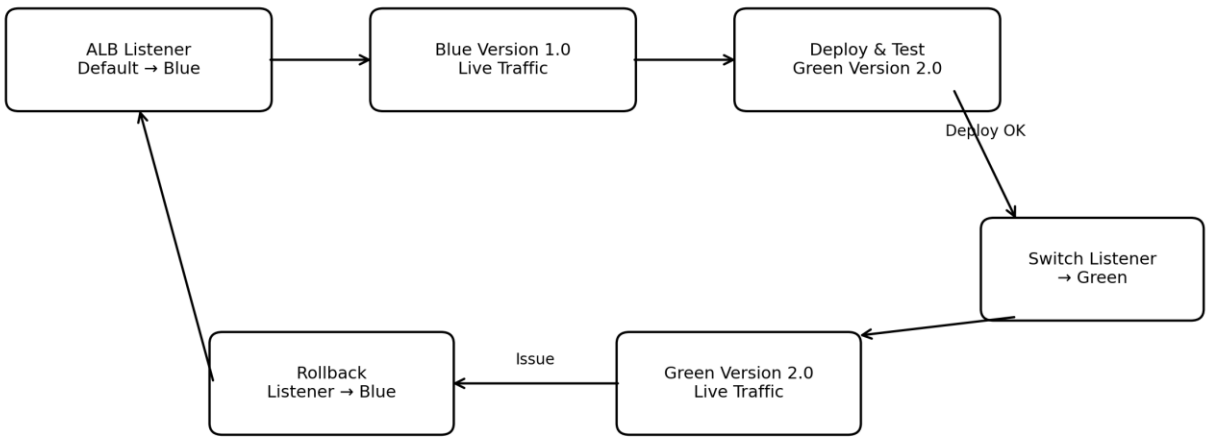


Figure 3: Traffic Switching and Rollback

8. PROJECT REQUIREMENTS

Hardware requirements:

- A computer/laptop with internet access.
- A modern web browser.
- An AWS account with permission to create the required resources.

Software/knowledge requirements:

- Basic Linux commands.
- Basic HTML/CSS knowledge.
- Basic AWS EC2 and networking knowledge.
- Understanding of HTTP and web servers.

9. IMPLEMENTATION METHODOLOGY

Step 1 – Prepare the Blue environment: Launch an Amazon Linux EC2 instance, install Apache, enable the service, and configure the Version 1.0 web page.

Step 2 – Create Blue Target Group: Create a target group using HTTP port 80 and register the Blue EC2 instance. Verify that the target becomes Healthy.

Step 3 – Prepare the Green environment: Launch a second Amazon Linux EC2 instance and configure Apache with the Version 2.0 web page.

Step 4 – Create Green Target Group: Create a second HTTP target group and register the Green EC2 instance. Verify its health status.

Step 5 – Create the ALB: Create an internet-facing Application Load Balancer across suitable subnets and attach the ALB security group.

Step 6 – Configure the listener: Create an HTTP listener on port 80 and initially forward traffic to Blue Target Group.

Step 7 – Test Blue: Open the ALB DNS name and confirm that Version 1.0 is displayed.

Step 8 – Deploy Green: Verify the Green target is healthy and ready for production traffic.

Step 9 – Switch traffic: Change the listener's default forwarding target from Blue Target Group to Green Target Group.

Step 10 – Validate and rollback: Confirm Version 2.0 is displayed. If the new release fails, switch the listener back to Blue.

10. CONFIGURATION DETAILS

10.1 Blue EC2 User Data

```
#!/bin/bash
dnf update -y
dnf install -y httpd
```

```
systemctl enable httpd
systemctl start httpd

cat > /var/www/html/index.html <<'EOF'
<!DOCTYPE html>
<html>
<head><title>Blue Environment</title></head>
<body>
<h1>BLUE ENVIRONMENT</h1>
<div>Version 1.0</div>
<p>Running on AWS EC2 + Apache</p>
</body>
</html>
EOF

systemctl restart httpd
```

10.2 Green EC2 User Data

```
#!/bin/bash
dnf update -y
dnf install -y httpd
systemctl enable httpd
systemctl start httpd

cat > /var/www/html/index.html <<'EOF'
<!DOCTYPE html>
<html>
<head><title>Green Environment</title></head>
<body>
<h1>GREEN ENVIRONMENT</h1>
<div>Version 2.0</div>
<p>New version deployed successfully!</p>
</body>
</html>
EOF

systemctl restart httpd
```

10.3 Security Group Design

Security Group	Inbound Rule	Source	Purpose
BlueGreen-ALB-SG	TCP 80	0.0.0.0/0	Allow public HTTP access to ALB.
BlueGreen-Instance-SG	TCP 80	BlueGreen-ALB-SG	Allow web traffic only from ALB.
BlueGreen-Instance-SG	TCP 22	My IP	Administrative SSH access when required.

11. BLUE-GREEN DEPLOYMENT PROCEDURE

1. Launch Blue EC2 in the selected VPC/subnet and attach the instance security group.
2. Use user data to install Apache and publish Version 1.0.
3. Confirm the Blue target is Healthy in Blue Target Group.
4. Launch Green EC2 using similar network settings.
5. Use user data to install Apache and publish Version 2.0.
6. Confirm the Green target is Healthy in Green Target Group.
7. Create an internet-facing ALB using at least two suitable subnets/AZs when available.
8. Configure HTTP port 80 listener with Blue Target Group as the initial default action.
9. Open the ALB DNS name and verify the Blue Version 1.0 page.
10. Edit the listener's default action and change the forwarding target to Green Target Group.

11. Open/refresh the ALB DNS name and verify Version 2.0.
12. For rollback, change the listener default action back to Blue Target Group.
13. After testing, remove resources that are no longer required to control AWS costs.

12. TESTING AND VALIDATION

Test Case	Expected Result	Status
Blue EC2 web server	Blue Version 1.0 page loads	Pass / Fail
Green EC2 web server	Green Version 2.0 page loads	Pass / Fail
Blue target health check	Target becomes Healthy	Pass / Fail
Green target health check	Target becomes Healthy	Pass / Fail
ALB → Blue	ALB shows Version 1.0	Pass / Fail
ALB → Green	ALB shows Version 2.0	Pass / Fail
Rollback	ALB returns to Version 1.0	Pass / Fail
Security validation	EC2 HTTP access is restricted to ALB SG	Pass / Fail

Testing should be performed only after target groups report healthy targets. If a target is unhealthy, common checks include the EC2 security group, Apache service status, port 80, health-check path, subnet/network configuration, and instance status.

13. SECURITY CONSIDERATIONS

- The Application Load Balancer is the public entry point; EC2 web access should be restricted to the ALB security group.
- SSH should be limited to the administrator's current IP rather than opened to the entire internet.
- Avoid unnecessary inbound rules such as unrestricted custom ports.
- Use separate security groups for the ALB and EC2 instances.
- Use IAM permissions with least privilege when automation requires AWS API access.
- Do not store passwords, private keys, or AWS access keys inside the website files or scripts.

14. ADVANTAGES AND LIMITATIONS

14.1 Advantages

- Low-risk release process because the new version is prepared separately.
- Fast rollback by changing the ALB forwarding target.
- Simple architecture that is easy to understand and demonstrate.
- Health checks help prevent traffic from being sent to unhealthy targets.
- Supports future gradual rollout strategies such as weighted traffic.

14.2 Limitations

- Two environments can temporarily require additional compute resources.
- Database migrations require careful backward/forward compatibility planning.
- This learning project uses simple static pages rather than a production application.
- Manual listener switching is used; production systems may automate deployment and validation.

15. COST AND RESOURCE MANAGEMENT

AWS costs depend on account eligibility, region, resource configuration, and current AWS pricing. For a learning project, resources should be created only for the duration of the lab. In particular, the Application Load Balancer and EC2 instances should be deleted/terminated after testing when they are not needed. Always review the AWS Billing and Cost Management console and the current AWS pricing pages before starting a long-running deployment.

- Use Free Tier/credit-eligible resources where the account is eligible.
- Stop or terminate unused EC2 instances as appropriate.
- Delete the ALB and target groups after the project demonstration.
- Remove unused security groups and other test resources.
- Monitor AWS billing alerts/budgets where available.

16. RESULTS

The expected result is a working Blue-Green deployment on AWS. The Application Load Balancer initially serves the Blue Version 1.0 application. After the Green Version 2.0 application is deployed and passes health checks, the ALB listener is changed to Green and user traffic reaches Version 2.0. If the Green version is considered unsuccessful, the listener can be changed back to Blue, demonstrating rapid rollback without rebuilding the original environment.

17. FUTURE ENHANCEMENTS

- Automate deployment using AWS CodeDeploy, CodePipeline, or another CI/CD solution.
- Use Launch Templates and Auto Scaling Groups instead of manually managed EC2 instances.
- Add HTTPS using an SSL/TLS certificate and an HTTPS listener.
- Add Amazon CloudWatch monitoring, logs, alarms, and dashboards.
- Use weighted target groups for gradual traffic shifting such as 90/10, 50/50, and 0/100.
- Use a database with a migration strategy designed for zero-downtime releases.
- Store application artifacts in Amazon S3 or a CI/CD artifact repository.
- Use Infrastructure as Code such as AWS CloudFormation or Terraform.

18. CONCLUSION

This project successfully demonstrates the core principles of Blue-Green Deployment using AWS. Two isolated application versions are deployed on separate EC2 instances, target groups provide health checking, and the Application Load Balancer controls which version receives user traffic. The ability to switch between environments and roll back quickly makes the deployment approach safer than directly replacing a running application. The project provides practical experience with AWS networking, EC2, Apache, security groups, load balancing, health checks, and DevOps release management.

19. REFERENCES

- Amazon EC2 User Guide – User data and launch-time configuration.
- Elastic Load Balancing User Guide – Application Load Balancers, listeners, target groups, and health checks.
- AWS Elastic Load Balancing Pricing – Load balancer and LCU pricing information.

- AWS IAM documentation – Identity and least-privilege access concepts.
- AWS VPC and Security Groups documentation – Network and traffic-control concepts.

